



EC-200 Data Structures

LAB MANUAL # 07

Course Instructor: Lecturer Anum Abdul Salam

Lab Engineer: Lab Engineer Ayesha Batool

Student Name: _____

Degree/ Syndicate: _____

	Trait	Obtained Marks	Maximum Marks
R1	Application Functionality 20%		20
R2	Specification & Data structure implementation 30%		30
R3	Reusability 10%		10
R4	Input Validation 10%		10
R5	Efficiency 20%		20
R6	Delivery 10%		10
R7	Plagiarism above 80%		1
Total			10

$$\text{Total Marks} = \text{Obtained Marks } (\sum_1^6 R_i * R_7)$$

Lab Manual # 07

Implementation of Queue

Lab Objective:

To implement Queue ADT using lists.

Lab Description:

A queue is a container in which insertions are made at end and deletions are done from start. Queue is a First in First out (FIFO) data structure. Unlike stacks, queue allows insertion from end and deletion from start. Like stacks queue can be implemented using

- i. Dynamic Approach (using Lists)
- ii. Static Approach (using arrays)

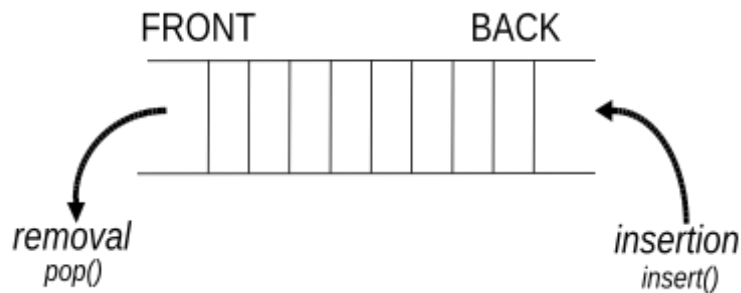


Figure 7.1: Queue Representation

Application of Queues:

1. In operating systems, for controlling access to shared system resources such as printers, files, communication lines, disks and tapes. A specific example of print queues follows:
 - a. In the situation where there are multiple users or a networked computer system, you probably share a printer with other users. When you request to print a file, your request is added to the print queue. When your request reaches the front of the print queue, your file is printed. This ensures that only one person at a time has access to the printer and that this access is given on a first-come, first-served basis.

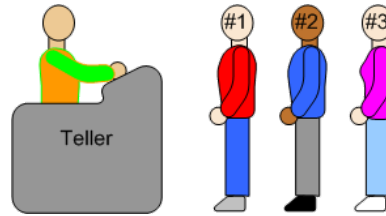


Figure 7.2: Queue application

2. For simulation of real-world situations. For instance, a new bank may want to know how many tellers to install. The goal is to service each customer within a "reasonable" wait time, but not have too many tellers for the number of customers. To find out a good number of tellers, they can run a computer simulation of typical customer transactions using queues to represent the waiting customers.
3. When data is transferred asynchronously (data not necessarily received at same rate as sent) between two processes. Examples include IO Buffers, pipes, file IO, etc.

Static Queue:

To implement a queue using array, create an array **arr** of size n and take two variables front and rear both of which will be initialized to 0 which means the queue is currently empty. Element rear is the index up to which the elements are stored in the array and front is the index of the first element of the array. Now, some of the implementation of queue operations are as follows:

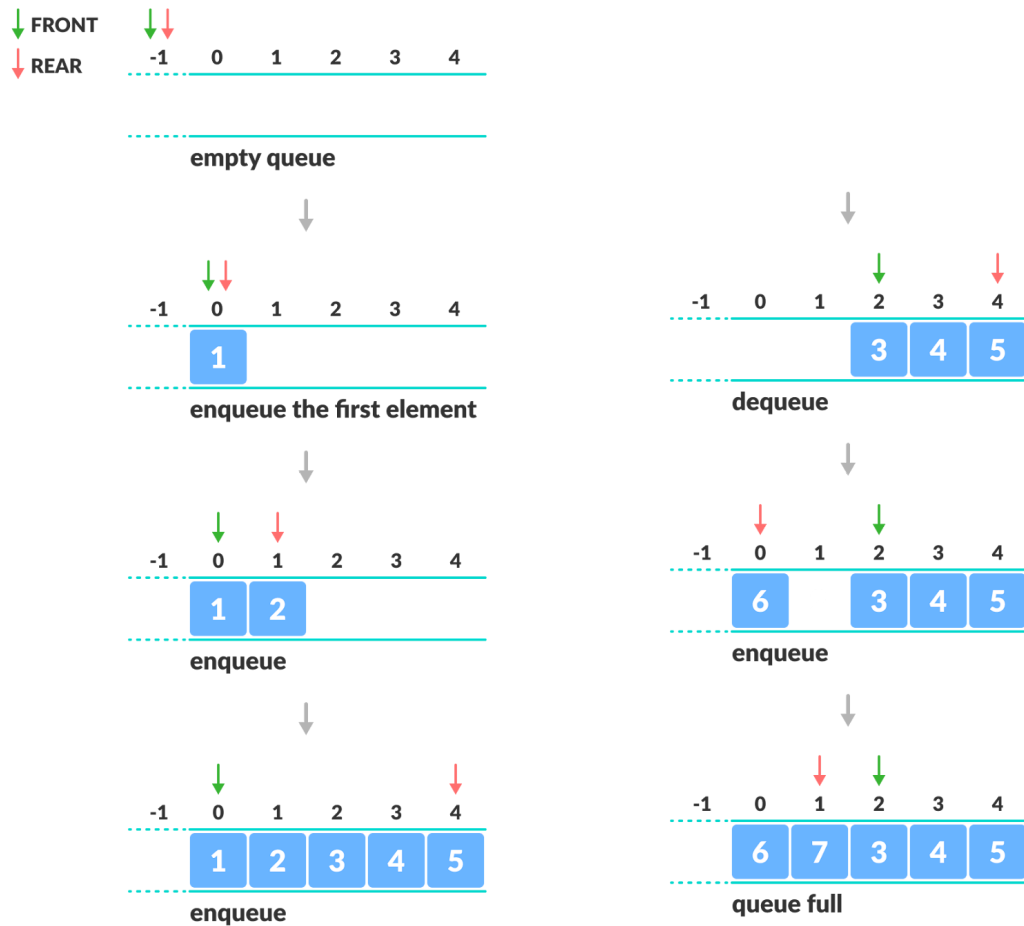
- **Enqueue:** Addition of an element to the queue. Adding an element will be performed after checking whether the queue is full or not. If $\text{rear} < n$ which indicates that the array is not full then store the element at $\text{arr}[\text{rear}]$ and increment rear by 1 but if $\text{rear} == n$ then it is said to be an Overflow condition as the array is full.
- **Dequeue:** Removal of an element from the queue. An element can only be deleted when there is at least an element to delete i.e., $\text{rear} > 0$. Now, element at $\text{arr}[\text{front}]$ can be deleted but all the remaining elements have to shifted to the left by one position in order for the dequeue operation to delete the second element from the left on another dequeue operation.
- **Front:** Get the front element from the queue i.e., $\text{arr}[\text{front}]$ if queue is not empty.
- **Display:** Print all element of the queue. If the queue is non-empty, traverse and print all the elements from index front to rear.

Circular Static Queue:

Circular Queue is a linear data structure in which the operations are performed based on FIFO (First in First Out) principle and the last position is connected back to the first position to make a circle. It is also called 'Ring Buffer'. In a normal Queue, we can insert elements until queue becomes full. But once queue becomes full, we cannot insert the next element even if there is a space in front of queue.

Operations on Circular Queue:

- **Front:** Get the front item from queue.
- **Rear:** Get the last item from queue.
- **enQueue(value)** This function is used to insert an element into the circular queue. In a circular queue, the new element is always inserted at Rear position.
 1. Check whether queue is Full – Check $((\text{rear} == \text{SIZE}-1 \ \&\& \ \text{front} == 0) \ || \ (\text{rear} == \text{front}-1))$.
 2. If it is full then display Queue is full. If queue is not full then, check if $(\text{rear} == \text{SIZE} - 1 \ \&\& \ \text{front} \neq 0)$ if it is true then set $\text{rear}=0$ and insert element.
- **deQueue()** This function is used to delete an element from the circular queue. In a circular queue, the element is always deleted from front position.
 1. Check whether queue is Empty means check $(\text{front} == -1)$.
 2. If it is empty then display Queue is empty. If queue is not empty then step 3
 3. Check if $(\text{front} == \text{rear})$ if it is true then set $\text{front} = \text{rear} = -1$ else check if $(\text{front} == \text{size}-1)$, if it is true then set $\text{front}=0$ and return the element.



Dynamic Queue:

Insertion and deletion of dynamic queue is same like insert at end (enqueue) and delete at start (dequeue) in single LinkedList.

Operations:

Queue ADT should provide following functions.

- i. Enqueue (insert at start)
- ii. Dequeue (delete from start)
- iii. Is Empty (to check is queue is empty)
- iv. Display (get front element of queue)
- v. Copy Constructor (for deep copying queue)
- vi. Destructor (De allocating queue memory)
- vii. Default constructor

LAB TASKS

1. Design templated Queue ADT using **static**, **circular static** and **dynamic** approach and provide all the above functionalities.

TEST PLAN (Static Queue):

1. Execute your test plan. If you discover mistakes in your implementation of the static queue ADT, correct them and execute your test plan again.

Sr.	Operations	Expected Results	Results/Status
1.	Create an empty queue SQueue<int> myQueue(5);	front=0; rear=0; size =5;	
2.	Insert numbers 1-5 in queue		
3.	Dequeue an element from myQueue	1	
4.	Enqueue 6 in myQueue		
5.	Dequeue an element from myQueue	2	
6.	Dequeue an element from myQueue	3	
7.	Dequeue an element from myQueue	4	
8.	Display myQueue	[5]	
9.	Dequeue an element from myQueue	5	
10.	Dequeue an element from myQueue	6	
11.	Enqueue 7 in myQueue		
12.	Enqueue 8 in myQueue		
13.	Display queue	[7]	

TEST PLAN (Circular Static Queue):

1. Execute your test plan. If you discover mistakes in your implementation of the circular static queue ADT, correct them and execute your test plan again.

Sr.	Operations	Expected Results	Results/Status
1.	Create an empty queue SQueue<float> myQueue(5);	front=0; rear=0; size=5;	

2.	Insert numbers 1-5 in queue		
3.	Dequeue an element from myQueue	1	
4.	Dequeue an element from myQueue	2	
5.	Enqueue 6 in myQueue		
6.	Enqueue 7 in myQueue		
7.	Display myQueue	[7]	
8.	Dequeue an element from myQueue	3	
9.	Dequeue an element from myQueue	4	
10.	Dequeue an element from myQueue	5	
11.	Enqueue 7 in myQueue		
12.	Enqueue 8 in myQueue		
13.	Display myQueue	[7]	

TEST PLAN (Dynamic Queue):

1. Execute your test plan. If you discover mistakes in your implementation of the dynamic queue ADT, correct them and execute your test plan again.

Sr.	Operations	Expected Results	Results/Status
1.	Create an empty queue SQueue<double> myQueue;	front=NULL rear=NULL	
2.	Insert numbers 1-5 in queue		
3.	Dequeue an element from myQueue	1	
4.	Enqueue 6 in myQueue		
5.	Dequeue an element from myQueue	2	
6.	Dequeue an element from myQueue	3	
7.	Dequeue an element from myQueue	4	
8.	Dequeue an element from myQueue	5	
9.	Dequeue an element from myQueue	6	
10.	Dequeue an element from myQueue	Queue empty	
11.	Enqueue 7 in myQueue		
12.	Enqueue 8 in myQueue		
13.	Display myQueue	[7]	

THINK:

1. What are some applications of Queues?
2. Why rear is maintained in queue to save address of last node?
3. What is the difference between queue and linked list?
4. What is difference between circular static queue and static queue?

